

TASK1

Q1

我只是初步了解了这些集合的结构以及用法

并了解了一下他们背后的数据结构和一些规则

个人认为其实没有完全彻底通透那些数据结构，数据结构那些规则堪比动物园怪谈

• Collection（单列集合）

特点：一次存一个数据

作用主要为集合的一些基本方法：**添加、删除、清空、判断、获取长度等**

◦ List

可以对指定位置增、删、改，返回

特点：添加元素有序（顺序和输入顺序相同）、可重复、有索引

■ ArrayList

底层的结构是**动态数组**

它的特点是元素有下标，**查询很快**；但是在中间插入、删除其他元素要挪位置，所以**增删较慢**

■ LinkedList

底层结构是**双向链表**

它的特点是不支持直接下标定位，**查询较慢**；但是在首尾处**增删快**

◦ Set

特点：元素无序（顺序和输入顺序不同）、不可重复、无索引

■ HashSet

底层结构基于HashMap

特点是无序，查找和添加极快，常用来去重

■ TreeSet

底层是红黑树

它常用来去重并对元素**进行按规则排序**

• Map（双列集合）

特点：一次存两个——对应数据，分别为不可重复的键(Key)，和可重复的值(Value)

◦ HashMap

底层结构是数组 + 链表 + 红黑树

存取是无序的，主要用来根据Key快速查找

◦ TreeMap

底层是红黑树

它的Key会自动**升序排序**

在需要按照Key进行排序的场景常用

• 数组与集合有什么区别？

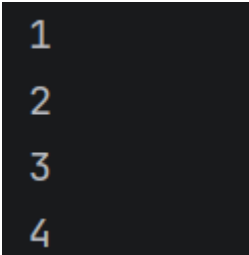
1. 长度上，数组一开始就确定了长度；而集合是动态的，长度可变
2. 存储数据上，数组只能存储一种类型的数据，且声明时就需要规定好；集合不使用泛型可以存多种类型数据，如需规定可以用泛型（虽然存多种不太好）
3. 数据类型上，数组可以存基本和引用数据类型变量；集合只能存引用类型变量，如果要存基本类型的只能用包装类
4. 功能与结构上，数组没有方法，结构是连续的内存数组；而集合有很多方法，也有很多除数组以外的结构

Q2

代码

```
import java.util.ArrayList;
import java.util.List;
public class Test {
    public static void main(String[] args) {
        List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);
        //按for(数据类型 临时变量名 : 集合/数组名)的格式来写
        //集合放的元素类型是Integer对象所以写Integer，这里也可以写int
        //把集合里的数依次传给i并依次打印
        for(Integer i:list){
            System.out.println(i);
        }
    }
}
```

结果截图



Q3

• 什么是匿名内部类？

匿名内部类就是没有名字的内部类。

一般来说，当一个抽象类或接口里面只有一个**抽象方法**的时候，如果再写一个实现类以用它的对象调用方法的话会略显麻烦。

此时就可以直接在方法体里面写一个匿名内部类，**直接充当一个实现类的实例对象**，节省代码

当然，有**多种抽象方法时也可以写匿名内部类**，但首先，这样写不利于代码的可读性；其次，**不能用lambda表达式简化写法**

它的格式如下：

```
new 类名/接口名() {  
    //对抽象方法的重写  
}
```

• 什么是函数式接口？

函数式接口是有且仅有一个抽象方法的接口

使用函数式接口，可以把数据处理的逻辑抽出来当成一个参数，**方便后面调用方法时自定义里面的逻辑**，使代码更灵活

比如常用的Consumer，它可以让一个方法不把方法体固定住，让关键的逻辑后面再传入（可以简化为lambda表达式）

例子：

有一个类Sales，里面需要一个输出打折后价格的方法

```
//Sales  
public void discount(Double p){  
    System.out.println(p*0.5);  
}
```

但是这样只能固定五折，所以可以用Consumer应对打六折七折这些情况

```
//Sales  
public void discount(Double p, Consumer<Double> consumer){  
    consumer.accept(p);  
}  
//Test(没有用lambda表达式，可读性有点差)  
public static void main(String[] args) {  
    //临时一个Sales实例对象调用discount方法，然后用匿名内部类写  
    new Sales().discount(100.0, new Consumer<Double>() {  
        @Override
```

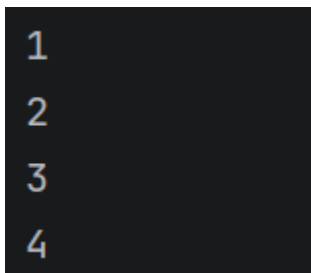
```
        public void accept(Double p) {  
            System.out.println(p*0.6);  
        }  
    });  
}
```

• 代码的改写

代码

```
//改写前  
list.forEach(new Consumer<Integer>() {  
    @Override  
    public void accept(Integer integer) {  
        System.out.println(integer);  
    }  
});  
  
//改写后  
list.forEach(integer -> System.out.println(integer));
```

结果截图



• lambda表达式的用法

在目标是一个函数式接口，也就是有且仅有一个抽象方法时，可以用它来简化代码，这在遍历等场景很常用
格式：(参数列表) -> { 方法体 }

可省略的内容：

- 参数类型可省略
- 只有一个参数时括号可省略
- 方法体只有一句时可省 {}和return

TASK2

Q4（个人说明在后面）

接口Repository

```
package generictest;
//定义一个Repository泛型接口
public interface Repository<T> {
    int save(T t); //save方法，保存数据并返回id
    T getById(int id); //根据id获取数据
}
```

实现类MyRepository

```
package generictest;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

public class MyRepository<T> implements Repository<T> {
    /*
    成员变量如下：
    1.由于一次要存两个值，用HashMap，key是整数，value为T
    2.一个id，初始值为0
    */
    private HashMap<Integer, T> repository = new HashMap<>();
    private int id=0;
    //重写save方法，用于保存数据，返回id
    @Override
    public int save(T t) {
        repository.put(id,t);
        //先用一个变量newid保存当前的id值再自增，不然和hashmap里的key对不上
        int newid = id;
        id++;
        //显然返回刚刚暂存的newid
        return newid;
    }

    //重写getById方法，用于根据id获取数据
    @Override
    public T getById(int id) {
        return repository.get(id);
    }

    //遍历方法getAll

    public void getAll() {
        /*
        这里用 获得集合的键值对对象 这个方法来遍历集合
        */
    }
}
```

1. `Map.Entry<Integer, T>`代表键值对对象，内部的泛型分别是`key`和`value`的类型
2. `Set`集合的泛型嵌套在刚刚泛型的外面，这个集合只能存键值对对象
3. `repository.entrySet()`获得集合的键值对对象封装为`Map.Entry`对象
返回一个`Set`集合赋值给`entries`

```

*/
Set<Map.Entry<Integer, T>> entries = repository.entrySet();
//遍历键值对对象，这里用增强for循环
for (Map.Entry<Integer, T> entry : entries) {
    //打印所有元素（id+内容）
    System.out.println("id:"+entry.getKey()+"->"+entry.getValue());
}
}
}

```

测试类Main

```

package genericstest;

public class Main {
    public static void main(String[] args) {
        //先放String类型的
        MyRepository<String> str_repo = new MyRepository<>();
        str_repo.save("微光娘");
        str_repo.save("Glimmer");
        str_repo.getAll();
        System.out.println("-----");

        //然后放User类型的
        User u1 = new User("微光娘",18);
        User u2 = new User("Ellery",18);
        MyRepository<User> user_repo = new MyRepository<>();
        user_repo.save(u1);
        user_repo.save(u2);
        user_repo.getAll();
        System.out.println("-----");

        //最后放Integer类型的
        MyRepository<Integer> int_repo = new MyRepository<>();
        int_repo.save(123);
        int_repo.save(456);
        int_repo.getAll();
    }
}

```

测试类Main的运行结果截图

```
id:0->微光娘
id:1->Glimmer
-----
id:0->User{name='微光娘', age=18}
id:1->User{name='Ellery', age=18}
-----
id:0->123
id:1->456
```

一些小说明

为什么直接用getValue而不用getKey放在getById?

我这里想练习一下这个键值对对象遍历，从视频里看的时候感觉看不懂就来写了一下（

我个人认为getById是外部查询内容才用到的，如果用在遍历确实不太恰当

我遇到的一些小卡顿

这里id我开始没管自增带来的影响

```
public int save(T t) {
    repository.put(id,t);
    id++;
    return id;
}
```

我key里放的是没有自增过的id，然后返回的是自增后的id，这样用返回后的id直接当key来查，结果就出问题了
ww

TASK 3

Q5

依旧前言与反思：

1. 关于三个实现类里我都没写get/set

我看这三个实现类都没有对成员变量操作的相关需求，只需要构造方法赋值一次，应该也不是javabean类，所以就没写

2. 关于构造方法

我一开始是写了一个带参构造的（没有审题而忽略了题目几个要求）

但是看到题目首先给了构造方法，其次还要求未来还可能扩展新的角色类型，那还是该成空参构造吧

这样以后创建新的角色类型就不需要那么麻烦了

3. 题目要求用的是枚举类，如果写的不对会爆Error，其实个人认为也有点多余了（不过题目有相关要求就写上吧）

4. 关于个人认为代码里可以改进的点

写完代码之后，个人发现有几个可以优化的点

1. if语句其实可以改成switchcase语句（由于作用相近，不再在题解里更改了）
2. 名字不能自定义（不过抽卡游戏应该确实也不行）

enum

```
public enum CharacterType {  
    //多整了个牧师用来测试其他未知类型  
    SABER, ARCHER, CASTER, PRIEST  
}
```

Character接口

```
public interface Character {  
    void attack();  
}
```

Saber类

```
public class Saber implements Character {  
    private String name;  
    private String skill;  
    //无参构造方法  
    public Saber(){  
        this.name = "阿尔托莉雅";  
        this.skill = "Excalibur";  
    }  
    //重写Character接口的attack方法  
    @Override  
    public void attack() {  
        System.out.println("[剑士] " + name + " 使用 " + skill + " 发动攻击! " );  
    }  
}
```

Archer类

```
public class Archer implements Character {  
    private String name;  
    private String skill;
```

```

//无参构造方法
public Archer(){
    this.name = "卫宫";
    this.skill = "Unlimited Blade works";
}
//重写Character接口的attack方法
@Override
public void attack() {
    System.out.println("[弓兵] " + name + " 使用 " + skill + " 发动攻击! " );
}
}

```

Caster类

```

public class Caster implements Character {
    private String name;
    private String skill;

    //无参构造方法
    public Caster(){
        this.name = "美狄亚";
        this.skill = "Rho Aias";
    }
    //重写Character接口的attack方法
    @Override
    public void attack() {
        System.out.println("[法师] " + name + " 使用 " + skill + " 发动攻击! " );
    }
}

```

CharacterFactory工厂类

```

public class CharacterFactory {
    //创建角色方法
    //工厂类里没有成员变量，所以直接用静态方法了
    public static Character createCharacter(CharacterType characterType) {
        //判断输入是否为空
        if (characterType == null) {
            throw new IllegalArgumentException("角色类型不能为空");
        }
        //下面判断角色类型
        if (characterType == CharacterType.SABER) {
            return new Saber();
        }else if (characterType == CharacterType.ARCHER) {
            return new Archer();
        }else if (characterType == CharacterType.CASTER) {
            return new Caster();
        }
        //如果都不是，抛出未知异常
    }
}

```

```
        throw new IllegalArgumentException("未知的角色类型"+characterType);
    }
}
```

测试类

```
public class Main {
    public static void main(String[] args) {
        //剑士
        Character ch1 = CharacterFactory.createCharacter(CharacterType.SABER);
        ch1.attack();
        //弓兵
        Character ch2 = CharacterFactory.createCharacter(CharacterType.ARCHER);
        ch2.attack();
        //法师
        Character ch3 = CharacterFactory.createCharacter(CharacterType.CASTER);
        ch3.attack();
        //null
        System.out.println("-----");
        System.out.println("若输入空值:");
        try {
            Character ch4 = CharacterFactory.createCharacter(null);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        //其他类型
        System.out.println("-----");
        System.out.println("若输入其他类型:");
        try{
            Character ch5 = CharacterFactory.createCharacter(CharacterType.PRIEST);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

测试类运行结果截图

[剑士] 阿尔托莉雅 使用 Excalibur 发动攻击!

[弓兵] 卫宫 使用 Unlimited Blade Works 发动攻击!

[法师] 美狄亚 使用 Rho Aias 发动攻击!

若输入空值:

角色类型不能为空

若输入其他类型:

未知的角色类型PRIEST